

Facturador Self-Hosted Perú

Manual del proyecto, teoría y glosario

Un manual para entender de qué se trata este software,
qué hace cada pieza, por qué fue construido así
y cómo usarlo aunque no sepas programar.

Versión inicial — 2026

Índice

1. Para qué sirve este software	pág. 4
2. Un poco de contexto: la facturación electrónica en Perú	pág. 5
3. Por qué self-hosted y no un servicio en la nube	pág. 7
4. Las piezas del sistema	pág. 8
5. El viaje de una factura, paso a paso	pág. 11
6. Conceptos técnicos explicados sin jerga	pág. 13
7. Cómo se configura todo	pág. 17
8. Tu primera factura beta	pág. 19
9. Cuando SUNAT rechaza: cómo leer el error	pág. 21
10. Pasar a producción	pág. 22
11. Mantenimiento, respaldos y certificado	pág. 23
12. Roadmap: qué falta y cuándo	pág. 24
13. Glosario de términos	pág. 25
14. Anexo A — Códigos de error SUNAT más comunes	pág. 30
15. Anexo B — Mapa de archivos del proyecto	pág. 32

1. Para qué sirve este software

Este proyecto resuelve un problema concreto: **en Perú, todo negocio formal está obligado a emitir comprobantes electrónicos (facturas, boletas, notas de crédito y notas de débito) y enviarlos a SUNAT**. Hacer eso a mano es imposible y las empresas comerciales que ofrecen ese servicio cobran mensualidades — entre S/ 30 y S/ 150 al mes — que se acumulan y que muchas veces no se justifican técnicamente.

Este software hace exactamente lo mismo que esos servicios comerciales (Nubefact, Efact, Facpro, Kesito y compañía), pero **corre en tu propia computadora o en un servidor que vos alquilás**. Vos sos el dueño de tus datos, de tu certificado, y de cuándo y cómo se emiten tus comprobantes. No pagás mensualidad porque no estás contratando un servicio, sino corriendo un programa.

La analogía más simple: usar un facturador comercial es como alquilarle a alguien una caja registradora y pagarle todos los meses por usarla. Este proyecto es como comprarte la caja registradora una sola vez — bueno, gratis en este caso — y ponerla en tu negocio.

Nota. Este software **no es** un PSE ni un OSE — no firmamos comprobantes en nombre de terceros ni custodiamos certificados de nadie. Es un programa que vos corrés, con tu propio certificado, en tu propia infraestructura.

Qué emite, qué no emite

Hoy emite:

- **Factura** (tipo 01): comprobante para ventas entre empresas con RUC.
- **Boleta de venta** (tipo 03): comprobante para personas naturales con DNI.
- **Nota de crédito** (tipo 07): corrección o devolución de una factura/boleta anterior.
- **Nota de débito** (tipo 08): aumento del monto a cobrar respecto a un comprobante anterior.

Hoy **no** emite:

- **Guía de Remisión Electrónica (GRE)**: queda para una versión posterior. SUNAT la exige obligatoriamente desde julio de 2026, así que está en el roadmap.
- **Comprobantes de retención y percepción**: aplicables si sos agente de retención o percepción.
- **Resúmenes diarios y comunicaciones de baja**: para anular boletas o anular comprobantes ya enviados.

Para quién es

Pensado para **micro, pequeñas y medianas empresas (MYPE)** que quieran soberanía sobre su facturación y no quieran pagar una suscripción. Concretamente:

- Un restaurante, ferretería, bodega, taller, consultorio, estudio contable — cualquier negocio que emita comprobantes con frecuencia.
- Un contador que quiera correr un servidor propio para emitir comprobantes de varios clientes (multi-RUC).

- Un dueño con dos o tres empresas que quiera centralizar la emisión.

No es para grandes contribuyentes (PRICOS) que facturen más de 300 UIT al año (S/ 1.650.000 al 2026): la ley les exige usar un OSE registrado, no auto-emisión.

2. Un poco de contexto: la facturación electrónica en Perú

Antes de meternos en cómo funciona el software, conviene entender qué es lo que SUNAT exige. Esto sirve para que después, cuando leas las partes técnicas, todo tenga sentido.

Qué es un Comprobante de Pago Electrónico (CPE)

En Perú, hace más de una década, SUNAT migró toda la facturación a formato electrónico. Un CPE es básicamente un **archivo XML** (un tipo de archivo de texto estructurado) que contiene los datos del comprobante: emisor, receptor, fecha, productos, montos, impuestos. Ese XML **tiene que estar firmado digitalmente** con un certificado del contribuyente, y **tiene que enviarse a SUNAT** en un formato técnico específico llamado SOAP.

SUNAT responde con otro archivo, llamado **CDR** (Constancia de Recepción): un ZIP que adentro tiene un XML que dice si tu comprobante fue aceptado, aceptado con observaciones, o rechazado. Ese CDR es la prueba legal de que el comprobante fue presentado.

Nota. Los tres archivos que importan para cada comprobante son: (1) el XML que enviaste, (2) el CDR que SUNAT te devolvió, y (3) la representación impresa (PDF) que le entregás al cliente. Los tres tienen que conservarse **mínimo 5 años**.

Los actores: emisor, OSE y PSE

SUNAT diseñó el sistema con tres roles posibles:

- **Emisor electrónico:** el contribuyente que emite. Vos. El que vende algo y debe entregar comprobante.
- **OSE (Operador de Servicios Electrónicos):** empresa autorizada por SUNAT que **valida** los comprobantes antes de que lleguen a SUNAT. Obligatorio para PRICOS grandes. Para MYPE y medianas no aplica.
- **PSE (Proveedor de Servicios Electrónicos):** empresa que ofrece la **infraestructura** de emisión como servicio. Aquí entran los Nubefact, Efact, etc. Cobran mensualidad.

Este software te permite operar como **emisor electrónico directo**, sin pasar por un PSE. SUNAT llama a esta modalidad **SEE del Contribuyente** (Sistema de Emisión Electrónica del Contribuyente).

Qué necesitás conseguir antes de usar el software

Hay tres cosas que solo vos podés conseguir, no las puede automatizar ningún software. Son trámites en SUNAT.

1. Tu RUC activo y habido

Si ya estás operando formalmente, ya lo tenés. Tu RUC debe estar en estado **activo** (no suspendido) y **habido** (SUNAT te puede ubicar en tu domicilio fiscal). Si no, no podés facturar electrónicamente ni con este software ni con ningún otro.

2. Tu Certificado Digital Tributario (CDT)

Es un **archivo .p12** que SUNAT te entrega gratis. Sirve para firmar digitalmente cada CPE que emitís. Lo bajás desde Clave SOL, en la sección de Comprobantes de Pago. Tiene vigencia 3 años. La passphrase del archivo la elegís vos al generarlo — anotala bien, sin ella el archivo no sirve. Mirá el documento *docs/obtener-cdt.md* en el repo para el paso a paso.

3. Un usuario secundario Clave SOL

SUNAT no quiere que uses tu Clave SOL principal (la que usás vos para entrar al portal) en un sistema automatizado. Te exige crear un **usuario secundario** con un permiso muy específico: *Emisión electrónica de comprobantes desde los sistemas del contribuyente*. Sin este usuario, SUNAT responde error 0111 a todo intento de envío. El procedimiento completo está en *docs/crear-usuario-secundario.md*.

Nota. Regla de oro: la Clave SOL principal NUNCA va al software. Si la ves en un archivo de configuración, eso es un bug de seguridad. Solo el usuario secundario, con permisos limitados, va al sistema.

Plazos legales que importan

- **3 días calendario** para enviar el CPE a SUNAT, contados desde el día siguiente a la fecha de emisión. Si pasa el plazo, SUNAT rechaza aunque el comprobante ya esté en manos del cliente.
- **5 años** mínimo de conservación de cada CPE y CDR.
- **UIT 2026: S/ 5.500.** La sanción por no emitir cuando debías es del 50% UIT (S/ 2.750) por comprobante, con rebaja del 90% si subsanás voluntariamente.

3. Por qué self-hosted y no un servicio en la nube

La pregunta natural es: **si Nubefact, Kesito, Efact y compañía ya existen y funcionan, ¿para qué construir esto?** La respuesta tiene tres partes.

Costo a lo largo del tiempo

Un facturador comercial cobra entre S/ 30 y S/ 150 mensuales. Eso son entre S/ 360 y S/ 1.800 al año, todos los años, sin parar. Para un negocio chico que emite pocas facturas es una carga significativa. Para un negocio que ya creció, es un gasto recurrente que no aporta más valor a medida que crece.

Un VPS donde correr este software cuesta entre US\$ 4 y US\$ 10 al mes (S/ 15 a S/ 38). Una laptop vieja conectada a internet puede correrlo gratis si tu volumen es bajo.

Soberanía sobre tus datos

Con un facturador comercial, **tus comprobantes, tu certificado y tus datos de clientes están en servidores de un tercero**. Eso significa que si la empresa quiebra, sube precios, te corta el servicio por una disputa, o tiene una brecha de seguridad, vos sufrís. Tu información se va con ellos.

Con self-hosting, tu certificado .p12 nunca sale de tu servidor. La passphrase que lo desbloquea solo existe en la memoria del proceso, nunca se guarda en disco. Los XMLs y CDRs los respaldás vos donde quieras. Si en algún momento querés cambiar de software, te llevás todo.

Transparencia técnica

El código es abierto. Vos podés leer (o pedirle a un técnico de confianza que lea) qué hace cada parte. Con un servicio comercial, tenés que confiar a ciegas en que están haciendo lo que dicen, cobrándote lo justo, y cumpliendo SUNAT bien.

La contra honesta

Self-hosting no es gratis en esfuerzo. Hay que:

- Instalar y configurar el software una vez.
- Mantener al día el servidor o la laptop donde corre.
- Hacer respaldos regulares de los CPE y CDR (responsabilidad legal).
- Renovar el certificado cada 3 años (gratis pero hay que acordarse).

Si nada de esto te interesa o no tenés cómo manejarlo, un facturador comercial sigue siendo una opción legítima. Este software es para quien valora la soberanía y el ahorro a largo plazo por sobre la comodidad de pagar mensualidad.

4. Las piezas del sistema

Un sistema de facturación no es una sola pieza, son varias. Pensalo como una cocina industrial: hay quien toma el pedido, quien cocina, quien empaca y quien entrega. En este software pasa lo mismo. Veamos cada pieza con esa analogía.

La cara visible: el frontend web

Es la **página que ves en el navegador** cuando entrás al sistema. Te muestra formularios para crear facturas, listados de comprobantes emitidos, configuración. Está construida en **React**, una tecnología muy difundida para hacer interfaces web. Es una PWA (Progressive Web App), lo que significa que **funciona también sin internet**: si se te cae la conexión, podés seguir armando facturas y el sistema las enviará a SUNAT cuando la conexión vuelva.

Analogía: el mesero del restaurante. Toma tu pedido y te muestra qué hay. No cocina nada, solo es el intermediario entre vos y la cocina.

El que decide: el backend API

Es el cerebro del sistema. Recibe los pedidos del frontend, **valida** que los datos sean correctos (por ejemplo, que un RUC tenga 11 dígitos, que las cantidades sean positivas), **recalcula los totales** y los impuestos (no se confía en lo que viene del navegador), genera el correlativo siguiente de la serie, y le pasa el trabajo al motor de facturación. Está escrito en **Go**, un lenguaje de programación moderno conocido por ser rápido y confiable.

Analogía: el jefe de cocina. Recibe la comanda, verifica que tenga sentido, organiza los ingredientes y le pasa la receta al cocinero especialista.

El especialista: el motor de facturación

Es la pieza que **habla el idioma de SUNAT**. Toma los datos del comprobante, los serializa en un XML con el formato exacto que SUNAT exige (un estándar llamado UBL 2.1), lo firma digitalmente con tu certificado, lo envuelve en un sobre técnico (SOAP), lo envía al servidor de SUNAT, y espera la respuesta (el CDR). Está escrito en **PHP** porque usa una librería peruana muy probada que se llama **Greenter**, que ya resuelve todos los detalles de la regulación SUNAT.

Analogía: el chef especializado en cocina molecular. Sabe exactamente cómo combinar ácido cítrico con esferas de calcio y montar el plato como exige el manual. El jefe de cocina no necesita saber esos detalles, solo le pasa el pedido.

La memoria: la base de datos

Es donde se **guardan permanentemente** los datos: tenants (empresas configuradas), usuarios del sistema, series y correlativos, comprobantes emitidos con su estado, bitácora de envíos a SUNAT. Es **PostgreSQL**, una base de datos robusta usada en miles de proyectos empresariales.

Analogía: el archivero del restaurante. Guarda todos los recibos, comandas, listas de proveedores, ventas del mes.

Nota. En la versión actual (MVP), la base de datos todavía no está totalmente activada — los comprobantes se guardan en archivos dentro de la carpeta `./data/`. La migración a PostgreSQL completa es uno de los próximos pasos.

La cola: Redis y el worker

SUNAT a veces está lento, a veces se cae, y nunca responde instantáneamente. Si cada vez que el cajero presiona *Emitir* tuviéramos que esperar a SUNAT antes de mostrarle algo, la experiencia sería pésima. La solución es una **cola**: cuando se emite, el comprobante se guarda y se mete en una lista de pendientes, y un proceso separado (el *worker*) los va sacando de a uno y enviando a SUNAT en segundo plano.

Redis es el software que mantiene esa cola en memoria.

Analogía: el sistema de tickets impresos en la cocina. Cuando entra un pedido, se cuelga en la línea; el cocinero los va tomando en orden mientras el mesero ya volvió a atender otra mesa.

Nota. En el MVP actual la emisión es **sincrónica** (el sistema espera a SUNAT antes de responder al cliente). La cola se activa en el próximo milestone. La pieza Redis ya está levantada en Docker, lista para cuando se conecte.

El orquestador: Docker Compose

Todas estas piezas (frontend, backend, motor, base de datos, Redis) son programas separados que tienen que arrancar en orden y comunicarse entre sí. **Docker** es una tecnología que las empaqueta cada una en su propio contenedor aislado. **Docker Compose** es la herramienta que coordina todos los contenedores con un solo comando.

Por eso al final, todo lo que tenés que hacer para arrancar el sistema es ejecutar:

```
docker compose -f docker-compose.yml -f docker-compose.beta.yml up -d
```

Y todo se levanta solo.

5. El viaje de una factura, paso a paso

Pongámonos en el ejemplo concreto. Vos atendés a un cliente y decidís emitir una factura. Esto es lo que pasa adentro del sistema, paso por paso. Te lo cuento como si fueras un auditor mirando desde afuera.

Paso 1 — Vos llenás el formulario

Abrís el frontend en el navegador, elegís el cliente o lo creás (RUC, razón social), agregás los ítems con cantidad y precio, ponés método de pago. El navegador te muestra el total calculado con IGV en tiempo real.

Paso 2 — El frontend manda los datos al backend

Cuando hacés click en **Emitir**, el frontend serializa todos esos datos como un JSON (otro formato de texto estructurado, más legible que XML) y lo envía al backend Go como una petición HTTP POST a la ruta `/api/v1/facturas`.

Paso 3 — El backend valida y recalcula

El backend Go revisa: ¿el RUC del receptor tiene 11 dígitos? ¿la fecha de emisión es válida? ¿los ítems tienen cantidad y precio positivos? ¿la serie corresponde al tipo de comprobante? Después, **recalcula todos los totales y el IGV desde cero**. El motivo es de seguridad y correctitud: si el frontend tuviera un bug o si alguien con malas intenciones modificara los valores enviados, no queremos terminar emitiendo un comprobante con un IGV calculado mal.

Paso 4 — Se asigna el correlativo

Cada serie (F001, F002, B001, etc.) tiene su propio contador. El backend toma el siguiente número de manera **atómica**: esto quiere decir que si llegaran dos emisiones al mismo tiempo, el sistema garantiza que no asigne el mismo número a ambas. Esto es crítico, porque SUNAT rechaza comprobantes duplicados.

Paso 5 — Se envía el trabajo al motor

El backend Go arma un JSON con toda la info y se lo manda al motor PHP por HTTP, dentro de la red privada de Docker. Junto con los datos del comprobante, le pasa el certificado ya descifrado en memoria y las credenciales del usuario secundario Clave SOL.

Paso 6 — El motor genera el XML, lo firma y lo envía

El motor PHP usa Greenter para construir el XML UBL 2.1 con el formato exacto que SUNAT exige. Después aplica la **firma digital XAdES-BES** con tu certificado — esto es lo que le da validez legal al documento. Después lo envuelve en un sobre SOAP con WS-Security (donde van las credenciales SOL), y lo manda al servidor de SUNAT (beta si estás probando, producción si ya estás operando en serio).

Paso 7 — SUNAT responde con el CDR

Después de unos segundos (a veces 1, a veces 10, a veces más), SUNAT responde. La respuesta puede ser:

- **Aceptado** (código 0, sin observaciones): todo perfecto.
- **Aceptado con observaciones** (código 0 + warnings): el comprobante es válido pero hay algo a corregir para próximas emisiones.

- **Rechazado** (código > 0): el comprobante no fue aceptado. Hay que corregir y reemitir.

Paso 8 — Se guardan los artefactos

El sistema guarda en disco:

- El XML firmado que se envió a SUNAT.
- El CDR (ZIP) que SUNAT devolvió.
- Un archivo JSON con el resumen y estado.

Estos tres archivos son tu prueba legal. SUNAT los puede pedir en una fiscalización. Hay que conservarlos 5 años.

Paso 9 — Se genera el PDF para el cliente

Con los mismos datos, el sistema arma una representación impresa bonita (A4 y/o ticket de 80mm) con tu logo, datos, y un código QR que contiene el hash del CPE. Eso es lo que le entregás al cliente — por mail, por WhatsApp, o impreso.

Nota. El PDF bonito está en el roadmap inmediato. El MVP actual todavía no lo genera, pero el XML y el CDR ya quedan guardados.

6. Conceptos técnicos explicados sin jerga

Esta sección desarma los términos técnicos más importantes para que cuando aparezcan en mensajes de error, documentación, o conversaciones con técnicos, sepas exactamente de qué se está hablando.

XML — el idioma de SUNAT

XML es un formato de archivo de texto que estructura información con etiquetas, parecido al HTML de las páginas web. Un fragmento simplificado:

```
<cac:AccountingSupplierParty>
  <cbc:CustomerAssignedAccountID>20123456789</cbc:CustomerAssignedAccountID>
  <cbc:AdditionalAccountID>6</cbc:AdditionalAccountID>
</cac:AccountingSupplierParty>
```

Lo importante: SUNAT exige que el XML tenga una estructura exacta. Cada etiqueta tiene que estar en su lugar. Cada campo tiene que cumplir reglas (longitudes, códigos válidos, etc.). Por eso usamos Greenter — porque construir este XML a mano es muy propenso a errores.

UBL 2.1 — el estándar internacional

UBL son las siglas de *Universal Business Language*. Es un estándar internacional para formatos de documentos comerciales (facturas, órdenes de compra, notas de envío). SUNAT lo adoptó y le agregó las particularidades peruanas (catálogos de impuestos, tipos de documento de identidad, códigos de afectación de IGV).

Cuando alguien dice *UBL 2.1*, está hablando de la versión del estándar que SUNAT exige. Otras ediciones (UBL 2.0, UBL 2.2) existen pero no aplican.

Firma digital y XAdES-BES

Cuando vos firmás un papel con tu mano, ese trazo es difícil de imitar y prueba que vos validaste el documento. La firma digital es el equivalente para archivos: un proceso matemático que toma el contenido del archivo y tu certificado (que es como tu identidad digital), y produce una **huella única** que se adjunta al archivo. Cualquiera puede verificar después que esa huella corresponde a tu certificado y que el archivo no fue modificado.

XAdES-BES (XML Advanced Electronic Signature - Basic Electronic Signature) es la variante específica de firma digital para XML que SUNAT exige. Define exactamente qué partes del XML se firman y cómo se incluye la firma adentro del mismo archivo.

Certificado .p12 y passphrase

Tu certificado digital es un archivo con extensión *.p12* (también llamado PKCS#12). Adentro tiene dos cosas: **tu llave privada** (un número enorme que solo vos conocés) y **tu certificado** (que enlaza esa llave con tu identidad fiscal). El archivo está encriptado con una passphrase que vos elegiste cuando lo generaste en SUNAT.

Para firmar un CPE el sistema necesita las dos cosas: leer el archivo y desbloquearlo con la passphrase. Por eso, este software:

- Pide la passphrase al arrancar (no la guarda en ningún archivo).

- Descifra el .p12 una vez y mantiene las claves en memoria.
- Si el servidor se reinicia, hay que volver a introducir la passphrase.

Esto es una decisión consciente de diseño: queremos que **en ningún momento la passphrase esté escrita en disco**. Si alguien roba el servidor apagado, tiene el .p12 pero no la passphrase, y por lo tanto no puede firmar comprobantes.

SOAP — el camión que lleva el XML a SUNAT

SOAP es un protocolo viejo (de los años 2000) que sigue siendo usado por sistemas gubernamentales y empresariales. Es como un sobre: adentro va tu XML firmado, y por fuera tiene un encabezado con datos de quién lo envía y cómo se autentica. SUNAT recibe ese sobre en una dirección específica de internet (un endpoint).

WS-Security: cómo se autentica el envío

WS-Security es la parte del sobre SOAP donde van las credenciales. Específicamente, SUNAT usa el modo *UsernameToken con PasswordText*: tu usuario (*RUC + usuario secundario*) y la contraseña en texto plano. ¿Texto plano no es inseguro? Sí, pero todo viaja por HTTPS (conexión cifrada), así que en la práctica está protegido. Es decisión de SUNAT, no nuestra.

CDR — la respuesta de SUNAT

CDR significa **Constancia de Recepción**. Es la respuesta que SUNAT te devuelve después de procesar el comprobante. Llega como un archivo ZIP que adentro tiene un XML (*ApplicationResponse*) con el estado:

- Código de respuesta (0 = aceptado, otros = rechazo o aviso).
- Descripción legible: *'La Factura número F001-1 ha sido aceptada'*.
- Observaciones (si las hay).

El CDR es la **prueba legal** de que SUNAT recibió el comprobante. Sin CDR, ante una fiscalización, no podés probar que enviaste.

Beta y producción: los dos mundos de SUNAT

SUNAT mantiene dos ambientes idénticos:

- **Beta (homologación)**: pensado para probar. Los comprobantes que envíes acá **no tienen efecto legal**. SUNAT los recibe, valida y responde igual que en producción, pero no quedan registrados como reales. Sirve para verificar que tu sistema funciona antes de emitir en serio.
- **Producción**: el real. Lo que envíes acá **sí tiene valor legal**. Hay que estar 100% seguro antes de tocarlo.

Este software permite cambiar entre ambos con una sola variable de configuración: *SUNAT_MODE=beta* o *SUNAT_MODE=prod*. Lo demás se mantiene igual.

IGV y la regla del 18%

El Impuesto General a las Ventas (IGV) en Perú es del 18% sobre el valor de venta. Pero no todos los ítems están afectos: hay productos exonerados (frutas, vegetales sin procesar), inafectos (servicios financieros), gratuitos, etc. SUNAT tiene una tabla (el catálogo 07) con los códigos de afectación. El sistema usa estos códigos para cada ítem y calcula el IGV correspondiente.

Multi-tenant: una instalación, varias empresas

Tenant significa *inquilino*: cada empresa (cada RUC) que usa el sistema es un tenant. Multi-tenant quiere decir que una sola instalación del software puede manejar varias empresas a la vez, manteniendo los datos de cada una completamente separados.

Por ejemplo, si sos contador y tenés cinco clientes, podés correr una sola instalación de este software y cargar los cinco RUCs con sus respectivos certificados. Cada cliente ve solo lo suyo.

MVP — Minimum Viable Product

MVP es un término del desarrollo de software. Significa *la versión más simple posible que ya hace algo útil*. La idea: en vez de construir todo de una y tardar un año, construís primero el flujo crítico (emitir UNA factura contra SUNAT beta) y a partir de ahí vas agregando funcionalidad. Lo que tenés ahora es el MVP.

7. Cómo se configura todo

Esta sección no es tutorial paso a paso de instalación — eso está en *docs/instalacion.md*. Acá explicamos **qué configura cada cosa y por qué importa**.

El archivo `.env`: el centro de control

Todo se configura en un archivo llamado `.env` que vos armás copiando `.env.example`. Adentro hay variables como esta:

```
TENANT_RUC=20123456789
TENANT_RAZON_SOCIAL=MI EMPRESA SAC
CERT_PASSPHRASE=loquesea123
SUNAT_MODE=beta
```

Cada variable controla algo específico. Las más importantes:

Identidad de tu empresa (`TENANT_*`)

- **`TENANT_RUC`**: tu RUC de 11 dígitos.
- **`TENANT_RAZON_SOCIAL`**: nombre legal de la empresa.
- **`TENANT_NOMBRE_COMERCIAL`**: nombre comercial (puede ser el mismo).
- **`TENANT_DIRECCION_FISCAL`**: dirección registrada en SUNAT.
- **`TENANT_UBIGEO`**: código de 6 dígitos del distrito según SUNAT.
- **`TENANT_DEPARTAMENTO`, `TENANT_PROVINCIA`, `TENANT_DISTRITO`**: en mayúsculas.

Estos datos deben coincidir **exactamente** con lo que SUNAT tiene registrado. Si la razón social difiere por una letra, SUNAT rechaza.

Credenciales de envío (`TENANT_USUARIO_SOL`, `TENANT_CLAVE_SOL`)

Estas son las credenciales del **usuario secundario** que creaste en Clave SOL. Para pruebas en beta podés usar el clásico `MODDATOS / MODDATOS`; para producción, sí o sí el usuario tuyo.

El certificado (`CERT_PATH`, `CERT_PASSPHRASE`)

El archivo `.p12` va físicamente en la carpeta `certs/` del proyecto. La variable `CERT_PATH` dice dónde encontrarlo dentro del contenedor Docker (por defecto `/app/certs/cert.p12`). La variable `CERT_PASSPHRASE` es la contraseña que vos elegiste al generar el `.p12`.

Nota. El archivo `.env` no se sube al repositorio de código (está excluido). Si lo subís por error, la passphrase queda expuesta y tenés que regenerar el certificado en SUNAT. Cuidalo como cuidás tu DNI.

Modo SUNAT (`SUNAT_MODE`)

Tiene dos valores posibles: *beta* o *prod*. Empezá siempre en beta. Solo cuando ya validaste que el flujo completo funciona, cambialo a prod.

Secretos del servidor (`API_JWT_SECRET`, `POSTGRES_PASSWORD`)

Son contraseñas internas del sistema que no se las das a nadie. Tienen que ser largas y aleatorias. Para generarlas podés correr `openssl rand -hex 32` en una terminal.

8. Tu primera factura beta

Asumiendo que ya:

- Tenés tu `.p12` en `certs/cert.p12`.
- Llenaste el `.env` con tus datos.
- Instalaste Docker.

Estos son los comandos que vas a correr, en orden, en una terminal dentro de la carpeta del proyecto:

1. Levantar todo el stack

```
docker compose -f docker-compose.yml -f docker-compose.beta.yml up -d
```

Docker descarga las imágenes, las arma, las arranca. La primera vez puede tardar varios minutos. Las siguientes son casi instantáneas.

2. Verificar que esté todo arriba

```
curl http://localhost:8080/health
```

Si responde algo como `{"status":"ok","sunat_mode":"beta"}`, el backend está funcionando.

```
curl http://localhost:8080/ready
```

Si responde `{"motor":"ok"}`, el motor PHP también está vivo y conectado.

3. Emitir la primera factura demo

```
./scripts/probar-factura.sh
```

Este script envía una factura con datos de prueba: serie *F001*, correlativo *1*, un solo ítem por *S/ 100* más IGV, total *S/ 118*.

4. Leer la respuesta

El script imprime la respuesta de SUNAT en pantalla y te dice dónde quedaron los archivos. Si todo salió bien, vas a ver algo así:

```
■ SUNAT aceptó tu primera factura beta.  
Mirá ./data/01-F001-00000001/
```

Adentro de esa carpeta, los tres archivos clave:

- **registro.json**: resumen del comprobante.
- **firmado.xml**: el UBL firmado que se envió a SUNAT.
- **cdr.zip**: la Constancia de Recepción.

Estos son los tres archivos que tenés que conservar 5 años según la ley.

5. Emitir otra (cambiando correlativo)

```
CORRELATIVO=2 ./scripts/probar-factura.sh
```

O cambiando la serie:

```
SERIE=F002 CORRELATIVO=1 ./scripts/probar-factura.sh
```


9. Cuando SUNAT rechaza: cómo leer el error

SUNAT rechaza con un código numérico y un mensaje. La regla general:

- **Códigos 1000-1999:** errores de estructura del XML (suelen ser bugs del software, no de tus datos).
- **Códigos 2000-2999:** errores de validación de datos (RUC inválido, fecha mal, totales no cuadran).
- **Códigos 3000-3999:** errores de validación SUNAT más estrictos (afectación de IGV mal asignada, montos negativos).
- **Códigos 4000+:** **warnings**, no rechazo. El comprobante igual queda aceptado pero hay algo a mejorar.

Cuando SUNAT rechaza, el campo *mensaje* casi siempre te dice exactamente qué está mal. El Anexo A al final de este manual lista los códigos más frecuentes con qué hacer ante cada uno.

Si el motor o la API no responden

A veces el problema no es SUNAT, sino algo local. Comandos útiles para diagnosticar:

```
docker compose ps
```

Muestra qué contenedores están arriba y cuáles no.

```
docker compose logs api motor
```

Muestra los logs de los servicios. Si algo falló, el motivo casi siempre aparece ahí.

```
docker compose restart api motor
```

Reinicia los servicios. Después de reiniciar, hay que volver a poner la *CERT_PASSPHRASE* en el *.env* (es a propósito).

10. Pasar a producción

Solo después de validar el flujo en beta. Idealmente, después de:

- Emitir varias facturas y boletas beta sin errores.
- Verificar que los datos del emisor coinciden con SUNAT.
- Tener creado el usuario secundario real (no MODDATOS).
- Tener un plan de respaldos.

Los pasos para el cambio:

- Apagar el stack: *docker compose down*.
- En *.env*: cambiar *SUNAT_MODE=beta* a *SUNAT_MODE=prod*.
- En *.env*: cambiar *TENANT_USUARIO_SOL* y *TENANT_CLAVE_SOL* por los del usuario secundario real (no MODDATOS).
- Levantar sin el override beta: *docker compose up -d*.
- Probar con una factura a un cliente conocido antes de abrirle el sistema al equipo.

Nota. Después de cualquier reinicio del contenedor, el sistema vuelve a leer el *.env* y por lo tanto la passphrase del certificado. Esto es a propósito: queremos que la passphrase esté en un archivo que vos controlás, no guardada permanentemente.

11. Mantenimiento, respaldos y certificado

Qué respaldar

- **Carpeta *data/*:** contiene todos los XML firmados y CDR. Es lo más importante. Sin esto, ante una fiscalización, no podés probar lo que emitiste.
- **Carpeta *certs/*:** contiene tu .p12. Si lo perdés, lo regenerás desde Clave SOL (gratis).
- **Archivo *.env*:** contiene tu configuración. Sin esto reconfigurás todo a mano.
- **Base de datos PostgreSQL:** cuando esté activa, hay que hacer dumps regulares con *pg_dump*.

Con qué frecuencia respaldar

Recomendación realista para un negocio chico:

- **Diario:** copia automática de *data/* a un disco externo o a un servicio en la nube cifrado.
- **Semanal:** copia completa de todo, idealmente fuera del servidor donde corre el sistema.

Cuándo renovar el certificado

El CDT vale 3 años. El sistema te debería avisar 30 días antes (esa funcionalidad está en el roadmap). El procedimiento es el mismo que cuando lo bajaste por primera vez (Clave SOL → Empresas → CDT → Generar), con una nueva passphrase. Reemplazás el .p12 en *certs/cert.p12* y actualizás *CERT_PASSPHRASE* en el *.env*.

Actualizar el software

El software va a tener releases. La idea es que actualizar sea tan simple como:

```
git pull
docker compose pull
docker compose up -d
```

Antes de actualizar producción, hacé respaldo de *data/*. Las versiones intentarán ser compatibles hacia atrás, pero respaldar siempre es la regla.

12. Roadmap: qué falta y cuándo

El estado actual es un MVP funcional. Lo que sigue, en orden aproximado de prioridad:

Próximo (semanas)

- Cola asíncrona Redis + worker (eliminar la espera sincrónica a SUNAT).
- Persistencia completa en PostgreSQL (correlativos atómicos, bitácora de envíos).
- Notas de crédito y débito.
- Boletas con resumen diario.
- PDF bonito con QR.
- Autenticación de usuarios JWT (multi-usuario por tenant).

Mediano plazo (meses)

- Interfaz web completa (hoy solo es scaffolding).
- Funcionamiento offline real con cola local en el navegador.
- Importación masiva desde Excel.
- Notificaciones por email al cliente.
- Reportes y exportes contables.
- Multi-tenant real con onboarding de nuevos RUCs desde la UI.

Antes de julio 2026

- Guía de Remisión Electrónica (GRE), que SUNAT exigirá obligatoriamente.

Futuro lejano

- Distribución como binarios nativos (sin Docker).
- Empaquetado para Windows/Mac/Linux con instalador gráfico.
- Posible app móvil para emisión rápida.

13. Glosario de términos

Términos técnicos, regulatorios y de software que aparecen en el proyecto, en la documentación SUNAT, o cuando hables con un técnico. Ordenados alfabéticamente.

Término	Qué significa, en cristiano
API	Conjunto de rutas web que un programa expone para que otros programas le hablen. En este proyecto, la API (escrita en Go) es lo que el frontend usa para emitir comprobantes.
Backend	La parte del software que corre en el servidor, fuera del navegador. Decide qué guardar, qué validar, qué responder. En este proyecto, el backend está escrito en Go.
Beta (SUNAT)	Ambiente de pruebas de SUNAT. Lo que enviás acá no tiene efecto legal. Sirve para validar tu sistema antes de pasar a producción.
Boleta	Comprobante para personas naturales (consumidores finales). Identificado con tipo 03. Serie empieza con B.
CDR	Constancia de Recepción. Archivo ZIP que SUNAT te devuelve después de que recibe tu comprobante. Adentro está el veredicto: aceptado, aceptado con observaciones, o rechazado. Es prueba legal.
CDT	Certificado Digital Tributario. Archivo .p12 que SUNAT entrega gratis. Sirve para firmar digitalmente los CPE. Vigencia 3 años.
Clave SOL	Las credenciales (usuario y contraseña) que SUNAT te da para acceder a sus servicios online. Tenés una principal (la del titular del RUC) y podés crear secundarias con permisos limitados.
Código de afectación IGV	Código del catálogo SUNAT 07 que define cómo afecta el IGV a un ítem: 10 gravado, 20 exonerado, 30 inafecto, etc.
Comprobante de Pago Electrónico (CPE)	Cualquiera de los documentos electrónicos que SUNAT obliga a emitir: factura, boleta, nota de crédito, nota de débito, guía de remisión.
Container (Docker)	Una caja aislada donde corre un programa. Tiene su propio sistema de archivos, su propia red interna, sus propias dependencias. Si se rompe, no afecta al resto.
Correlativo	Número secuencial dentro de una serie. F001-1, F001-2, F001-3... SUNAT exige que sean consecutivos sin saltos.
Docker	Tecnología para empaquetar programas con todas sus dependencias y correrlos de manera aislada en cualquier servidor.

Término	Qué significa, en cristiano
Docker Compose	Herramienta de Docker para describir varios contenedores que se coordinan entre sí en un solo archivo YAML.
Emisor electrónico	El contribuyente que emite comprobantes. Vos.
Endpoint	Una dirección de internet a la que un programa le pide algo. SUNAT tiene endpoints distintos para beta y producción.
env (archivo .env)	Archivo de configuración con variables como TENANT_RUC, CERT_PASSPHRASE, etc. Cada programa que lo necesita las lee.
Factura	Comprobante para operaciones entre empresas o con consumidores con RUC. Tipo 01. Serie empieza con F.
Firma digital	Proceso matemático que prueba que un archivo fue validado por el dueño de un certificado y que el contenido no fue alterado.
Frontend	La parte del software que ves en el navegador. En este proyecto, hecha con React.
Git	Sistema para versionar código fuente. Permite ver el historial de cambios, volver atrás, trabajar en ramas paralelas.
Go	Lenguaje de programación creado por Google. Compilado, rápido, popular para servidores. Usado en el backend de este proyecto.
Greenter	Librería peruana de código abierto que resuelve la generación de UBL, firma XAdES, envío SOAP a SUNAT y parseo de CDR. Es el corazón del motor de facturación.
GRE	Guía de Remisión Electrónica. Documento que acompaña el traslado físico de mercadería. Obligatoria desde julio 2026.
Hash	Una huella única calculada matemáticamente sobre un archivo. Si cambia un solo carácter, el hash es completamente diferente. SUNAT incluye el hash del CPE en el código QR.
HTTP / HTTPS	El protocolo que usan los navegadores y APIs para comunicarse. HTTPS es la versión cifrada.
ICBPER	Impuesto al Consumo de Bolsas Plásticas. S/ 0.50 por bolsa (al 2026, sujeto a cambio).
IGV	Impuesto General a las Ventas. 18% sobre el valor de venta de bienes y servicios gravados.
ISC	Impuesto Selectivo al Consumo. Aplica a productos específicos (cigarrillos, alcohol, combustibles).

Término	Qué significa, en cristiano
JSON	Formato de texto para intercambiar datos. Más simple y legible que XML. Usado entre el frontend y el backend.
JWT	JSON Web Token. Tipo de credencial que el backend emite cuando un usuario se autentica. Sirve para que las siguientes peticiones se identifiquen sin pedir contraseña otra vez.
Migración (DB)	Archivo SQL que describe cómo crear o modificar tablas de la base de datos. Se aplican en orden cuando se actualiza el sistema.
MODDATOS	Usuario clásico de pruebas SUNAT para el ambiente beta. Sirve para que cualquiera pueda probar el flujo sin haber creado su usuario secundario aún.
Monorepo	Estructura de proyecto donde varios componentes (backend, frontend, motor) viven en un solo repositorio Git.
Motor de facturación	En este proyecto, el microservicio PHP que genera UBL, firma y envía a SUNAT. Aislado del resto por seguridad.
MVP	Minimum Viable Product. La versión más simple que ya hace algo útil.
Nota de crédito	Documento que reduce o corrige una factura/boleta previa (descuentos, devoluciones, anulaciones). Tipo 07.
Nota de débito	Documento que aumenta el monto de una factura/boleta previa (intereses, aumento de valor). Tipo 08.
OSE	Operador de Servicios Electrónicos. Empresa autorizada por SUNAT que valida comprobantes antes de SUNAT. Obligatorio para PRICOS grandes.
PEM	Formato de texto para representar certificados y llaves criptográficas. Empieza con -----BEGIN CERTIFICATE----- o similar.
PHP	Lenguaje de programación usado en el motor de facturación de este proyecto. Lo usamos porque Greenter, la librería peruana de facturación, está escrita en PHP.
PKCS#12	Estándar para empaquetar certificados con su llave privada en un solo archivo cifrado. Extensión .p12 o .pfx.
PostgreSQL	Base de datos relacional de código abierto, robusta. Usada en miles de proyectos empresariales. Es la DB de este proyecto.
PRICO	Principal Contribuyente. Empresas grandes designadas por SUNAT, con exigencias regulatorias más estrictas (OSE obligatorio).

Término	Qué significa, en cristiano
Producción (SUNAT)	Ambiente real de SUNAT. Lo que enviás tiene valor legal pleno.
PSE	Proveedor de Servicios Electrónicos. Empresa que ofrece la infraestructura de emisión como servicio. Nubefact, Efact, etc., son PSE.
PWA	Progressive Web App. Página web que también funciona instalada como app, con soporte offline.
Queue (cola)	Lista de tareas pendientes que se procesan una por una en segundo plano.
React	Librería de JavaScript para construir interfaces de usuario en el navegador. Usada en el frontend.
Redis	Base de datos en memoria, muy rápida. Usada para mantener la cola de jobs.
Reverse proxy	Programa que recibe pedidos de internet y los reparte a los servicios internos. Caddy y Nginx son ejemplos comunes. Recomendado si exponés el sistema a internet.
RUC	Registro Único de Contribuyente. Tu identificación tributaria en Perú. 11 dígitos.
Self-hosted	Software que vos corrés en tu propia infraestructura, en oposición a SaaS donde lo corre un tercero.
SEE	Sistema de Emisión Electrónica. El conjunto de modalidades por las que se emiten CPE. SEE del Contribuyente es la que usamos acá.
Serie	Identificador de 4 caracteres que agrupa un rango de correlativos. F001, F002 para facturas; B001, B002 para boletas.
SOAP	Protocolo de comunicación basado en XML. SUNAT lo usa para recibir los CPE.
SQL	Lenguaje para consultar y modificar bases de datos relacionales.
SSL/TLS	Las tecnologías que cifran el tráfico entre tu computadora y un servidor. Es lo que hace HTTPS seguro.
Stack	El conjunto de tecnologías que componen un proyecto. El stack de este proyecto es: Go + React + PHP + PostgreSQL + Redis + Docker.
Stateless	Que no guarda estado entre llamadas. El motor PHP de este proyecto es stateless: no tiene DB, si se reinicia no perdió nada.

Término	Qué significa, en cristiano
SUNAT	Superintendencia Nacional de Aduanas y de Administración Tributaria. El ente recaudador de Perú.
Tenant	En multi-tenant, cada empresa (RUC) que usa el sistema. Una instalación, varios tenants.
UBL	Universal Business Language. Estándar internacional para documentos comerciales en XML. SUNAT usa UBL 2.1.
Ubigeo	Código de 6 dígitos que SUNAT usa para identificar el departamento, provincia y distrito en Perú.
UIT	Unidad Impositiva Tributaria. Valor de referencia que se actualiza cada año. Al 2026: S/ 5.500. Se usa para calcular sanciones y umbrales.
Usuario secundario Clave SOL	Usuario adicional con permisos limitados creado bajo tu Clave SOL principal. El que se le da al software para que emita en tu nombre.
Volumen (Docker)	Carpeta del servidor host que se monta dentro de un contenedor. Permite que los datos sobrevivan reinicios del contenedor.
VPS	Virtual Private Server. Servidor virtual alquilado por mes. Una opción común para correr este software (DigitalOcean, Linode, Hetzner, etc.).
Worker	Proceso que corre en segundo plano sacando tareas de una cola y ejecutándolas. En este proyecto, el worker es el que envía a SUNAT.
WS-Security	Estándar para incluir credenciales y firmas en mensajes SOAP. SUNAT lo usa para autenticar el envío.
XAdES-BES	Variante de firma digital específica para XML que SUNAT exige. BES significa Basic Electronic Signature.
XML	Formato de texto estructurado con etiquetas. El idioma en el que viaja la información a SUNAT.
YAML	Formato de texto para configuración, más legible que XML o JSON. Docker Compose usa YAML.

14. Anexo A — Códigos de error SUNAT más comunes

Cuando SUNAT rechaza, devuelve un código numérico y un mensaje. Esta tabla cubre los más frecuentes:

Código	Significado	Qué hacer
0111	El usuario SOL no tiene perfil para enviar comprobantes electrónicos.	Crear o asignar permiso 'Emisión electrónica de comprobantes desde los sistemas del contribuyente' al usuario secundario.
0150	Las credenciales SOL son incorrectas.	Verificar <code>TENANT_USUARIO_SOL</code> y <code>TENANT_CLAVE_SOL</code> en <code>.env</code> . Recordar que el usuario va sin el RUC adelante en este software (el sistema lo concatena solo).
1032	El número de documento de identidad no es válido para el tipo indicado.	Revisar el RUC (11 dígitos), DNI (8), CE, etc., y que el tipo de documento (catálogo 06) corresponda.
1078	La factura debe identificar al cliente con RUC.	Cambiar a boleta o registrar el RUC del cliente.
2017	El número de RUC del receptor no existe.	Verificar el RUC. Puede haber un dígito mal.
2018	El RUC del receptor no está activo o no está habido.	El cliente debe regularizar su situación en SUNAT antes de poder recibir factura.
2335	El RUC del emisor no está activo.	Tu RUC propio está suspendido o de baja. Regularizar primero en SUNAT.
2800	Comprobante ya fue presentado anteriormente.	Ya enviaste este correlativo. No volver a enviar — buscar el CDR original.
3105	Los montos no cuadran con los ítems.	Generalmente un bug de cálculo. Reportar al desarrollador. El software recalcula server-side; este error implica un caso no contemplado.
3206	La fecha de emisión es posterior a la fecha actual.	Verificar la fecha del servidor (zona horaria) y la fecha enviada.
3211	Se excedió el plazo de envío (3 días calendario).	Comprobantes vencidos no pueden enviarse. Anular y emitir uno nuevo con fecha actual.

Código	Significado	Qué hacer
4000-4999	Warnings (no rechazos).	El comprobante quedó aceptado pero hay observaciones. Revisar para próximas emisiones.
Connection timeout	SUNAT no responde a tiempo.	Reintentar luego. La cola asíncrona maneja esto automáticamente cuando esté activada.

15. Anexo B — Mapa de archivos del proyecto

Si abris el repositorio, esto es lo que encontrás y qué hace cada cosa:

```
facturador/
■■■■ CLAUDE.md Contexto del proyecto (para IA y humanos)
■■■■ README.md Resumen y filosofía
■■■■ docker-compose.yml Define los servicios del stack
■■■■ docker-compose.beta.yml Override para forzar modo beta
■■■■ .env.example Plantilla de configuración
■
■■■■ apps/
■ ■■■■ api/ Backend Go
■ ■ ■■■■ cmd/api/main.go Punto de entrada del programa
■ ■ ■■■■ internal/
■ ■ ■ ■■■■ cert/ Carga del .p12, descifrado en memoria
■ ■ ■ ■■■■ config/ Lee variables de entorno
■ ■ ■ ■■■■ facturacion/ Modelo de factura y recálculo de totales
■ ■ ■ ■■■■ http/ Rutas y handlers HTTP
■ ■ ■ ■■■■ motor/ Cliente que habla con el motor PHP
■ ■ ■ ■■■■ storage/ Persistencia en filesystem (por ahora)
■ ■ ■ ■■■■ sunat/ Endpoints SUNAT por modo
■ ■ ■ ■■■■ tenant/ Configuración del tenant
■ ■ ■ ■■■■ db/migrations/ Scripts SQL para PostgreSQL
■ ■ ■■■■ Dockerfile Cómo construir la imagen Docker
■ ■
■ ■■■■ motor/ Motor PHP/Greenter
■ ■ ■■■■ src/Emisor.php Lógica de emisión
■ ■ ■■■■ public/index.php Servidor HTTP del motor
■ ■ ■■■■ composer.json Dependencias PHP
■ ■ ■■■■ CONTRACT.md Contrato entre Go y motor PHP
■ ■ ■■■■ Dockerfile
■ ■
■ ■■■■ web/ Frontend React PWA
■ ■■■■ src/App.tsx Componente principal
■ ■■■■ package.json Dependencias JavaScript
■ ■■■■ vite.config.ts Configuración del bundler
■ ■■■■ Dockerfile
■
■■■■ packages/
■ ■■■■ codigos-sunat/ Catálogos SUNAT (en construcción)
■ ■■■■ ubl-schemas/ Esquemas XSD UBL 2.1
■
■■■■ docs/
■ ■■■■ adr/0001-stack-motor.md Por qué Go + PHP
■ ■■■■ instalacion.md Guía paso a paso
■ ■■■■ obtener-cdt.md Cómo bajar el .p12 de SUNAT
■ ■■■■ crear-usuario-secundario.md Cómo crear el usuario SOL secundario
■ ■■■■ manual.py Script que genera este PDF
■ ■■■■ manual.pdf Este documento
■
■■■■ scripts/
■ ■■■■ probar-factura.sh Emite 1 factura beta de prueba
■ ■■■■ seed-beta.sh (Por implementar) crea tenant de prueba
■
■■■■ certs/ Tus certificados .p12 (no commiteado)
■■■■ data/ XML firmados y CDR (no commiteado)
```

Cualquier archivo que termina en *.go* es código del backend Go. Cualquier *.php* es del motor. *.tsx* y *.ts* son del frontend. *.sql* son migraciones de base de datos. *.yml* son configuraciones de Docker Compose. *.md* son documentos como este.

Cierre

Si llegaste hasta acá, ya tenés una idea general bastante completa de qué es este proyecto, cómo funciona y cómo se usa. No hace falta que entiendas el código línea por línea; alcanza con tener un mapa mental de las piezas y cómo conversan entre sí.

El objetivo del proyecto es muy concreto: **que un contribuyente peruano pueda emitir comprobantes electrónicos sin pagar mensualidad y sin perder control sobre sus datos.** Todo lo demás — código en Go, motor PHP, Docker, React — son medios para ese fin.

Cuando algo no te quede claro, o cuando aparezca un mensaje de SUNAT que no entiendas, el glosario y el Anexo A deberían cubrir la mayoría de los casos. Y si no, siempre podés consultar.

— Fin del manual —